

Padrões de Projeto no Sistema de Cardápio Digital “Sushi Wabi-Sabi”

1. Padrão State

Conceito

Permite que um objeto altere seu comportamento quando seu estado interno muda, como se ele tivesse “trocado de classe”. O estado é encapsulado em objetos separados.

Onde aparece no projeto

O **estado do pedido** na cozinha (pedidos.status) é o exemplo mais claro. Os possíveis estados são:

- pendente
- preparando
- concluído
- cancelado

Como é implementado (sem uma classe State explícita, mas com lógica condicional)

No arquivo **cozinha.php**, a renderização de cada pedido muda conforme o status:

```
if (pedido.status === 'pendente') {  
    // exibe botão "Iniciar"  
} elseif (pedido.status === 'preparando') {  
    // exibe botão "Concluir"  
} elseif (pedido.status === 'concluido') {  
    // não mostra botões de ação  
}
```

E na API **api_pedidos.php**, a atualização de status apenas altera o campo no banco e registra o log:

```
$sql = "UPDATE pedidos SET status = ? WHERE id_pedido = ?";  
// e depois insere um log com status_anterior e status_novo
```

Embora não haja uma classe **State** formal, o comportamento do sistema (quais botões aparecem, quais transições são permitidas) **depende inteiramente do valor do campo status**. Isso caracteriza o padrão **State** – a lógica de transição está espalhada, mas o princípio está presente.

Benefício

- Separa as ações possíveis para cada estado.
- Facilita adicionar novos estados (ex.: entregue) sem alterar toda a lógica.

- O histórico de mudanças (pedido_status_log) funciona como um “memento” do estado.

2. Padrão Observer

Conceito

Define uma dependência “um-para-muitos” entre objetos, de modo que quando um objeto muda de estado, todos os seus dependentes são notificados automaticamente.

Onde aparece no projeto

O sistema de **atualização automática da cozinha** é uma aplicação do Observer. Existem dois elementos:

- **Sujeito (observado):** a tabela pedidos no banco de dados (quando um novo pedido é inserido ou um status é alterado).
- **Observadores (interessados):** o painel da cozinha (`cozinha.php`) e, indiretamente, o cardápio (quando finaliza um pedido, ele notifica a cozinha).

Como é implementado

O padrão não é implementado com eventos nativos ou WebSockets; em vez disso, usa-se **polling (consulta periódica)**. No cozinha.php, a cada 10 segundos o JavaScript chama a API:

```
setInterval(() => {  
    const activeTab = document.querySelector('#pedidosTab .nav-link.active');  
    if (activeTab && activeTab.id === 'ativos-tab') {  
        carregarPedidosAtivos(); // busca pedidos pendentes/preparando  
    }  
}, 10000);
```

Quando a API retorna novos pedidos ou mudanças de status, a interface é atualizada. Além disso, um **som de notificação** (playNotificationSound()) é disparado quando um novo pedido chega (identificado pela mudança no ultimoPedidoId).

No lado do cliente (cardápio), quando um pedido é finalizado, a função confirmarPedido envia os dados para api_pedidos.php?action=salvar_pedido. Esse ponto age como uma “notificação” para o servidor, que por sua vez insere o pedido no banco. A cozinha, ao consultar o banco, reage a essa inserção.

Benefício

- Desacopla o momento da criação do pedido da sua exibição na cozinha.
- Permite que vários observadores (futuramente, um painel de delivery) escutem as mesmas mudanças.
- O polling é uma forma simples de implementar o padrão sem infraestrutura adicional.

Outros padrões notáveis

Padrão	Exemplo no projeto
Singleton	A conexão PDO (conexao.php) é uma única instância compartilhada.
Template Method	O <code>openCustomizeModal</code> possui um fluxo base (bebidas vs. comidas) com variações específicas para combinados.
Memento	O <code>localStorage</code> do carrinho armazena o estado do pedido e pode restaurá-lo.
Strategy	As formas de pagamento (Dinheiro, Pix, Cartão) são estratégias diferentes que alteram o cálculo do desconto.

Conclusão

O projeto **Sushi Wabi-Sabi** utiliza o padrão **State** para controlar o ciclo de vida dos pedidos na cozinha, e o padrão **Observer** (via polling e notificações sonoras) para manter o painel da cozinha atualizado em tempo quase real. Apesar de não usar implementações clássicas (classes State explícitas ou eventos pub/sub), os princípios estão claramente presentes, mostrando uma arquitetura preocupada com a separação de responsabilidades e a reatividade.